



[Home](#) / [Get started](#) / What is Docker?

What is Docker?

Table of contents

The Docker platform

What can I use Docker for?

Fast, consistent delivery of your applications

Responsive deployment and scaling

Running more workloads on the same hardware

Docker architecture

The Docker daemon

The Docker client

Docker Desktop

Docker registries

Docker objects

The underlying technology

Next steps

Docker is an open platform for developing, shipping, and running applications. Docker enables you to separate your applications from your infrastructure so you can deliver software quickly. With Docker, you can manage your infrastructure in the same ways you manage your applications. By taking advantage of Docker's methodologies for shipping, testing, and deploying code, you can significantly reduce the delay between writing code and running it in production.

The Docker platform

Docker provides the ability to package and run an application in a loosely isolated environment called a container. The isolation and security let you run many containers simultaneously on a given host. Containers are lightweight and contain everything needed to run the application, so you don't need to rely on what's installed on the host. You can share containers while you work, and be sure that everyone you share with gets the same container that works in the same way.

Docker provides tooling and a platform to manage the lifecycle of your containers:

- Develop your application and its supporting components using containers.
- The container becomes the unit for distributing and testing your application.
- When you're ready, deploy your application into your production environment, as a container or an orchestrated service. This works the same whether your production environment is a local data center, a cloud provider, or a hybrid of the two.

What can I use Docker for?

Fast, consistent delivery of your applications

Docker streamlines the development lifecycle by allowing developers to work in standardized environments using local containers which provide your applications and services.

Containers are great for continuous integration and continuous delivery (CI/CD) workflows.

Consider the following example scenario:

- Your developers write code locally and share their work with their colleagues using Docker containers.
- They use Docker to push their applications into a test environment and run automated and manual tests.
- When developers find bugs, they can fix them in the development environment and redeploy them to the test environment for testing and validation.
- When testing is complete, getting the fix to the customer is as simple as pushing the updated image to the production environment.

Responsive deployment and scaling

Docker's container-based platform allows for highly portable workloads. Docker containers can run on a developer's local laptop, on physical or virtual machines in a data center, on

cloud providers, or in a mixture of environments.

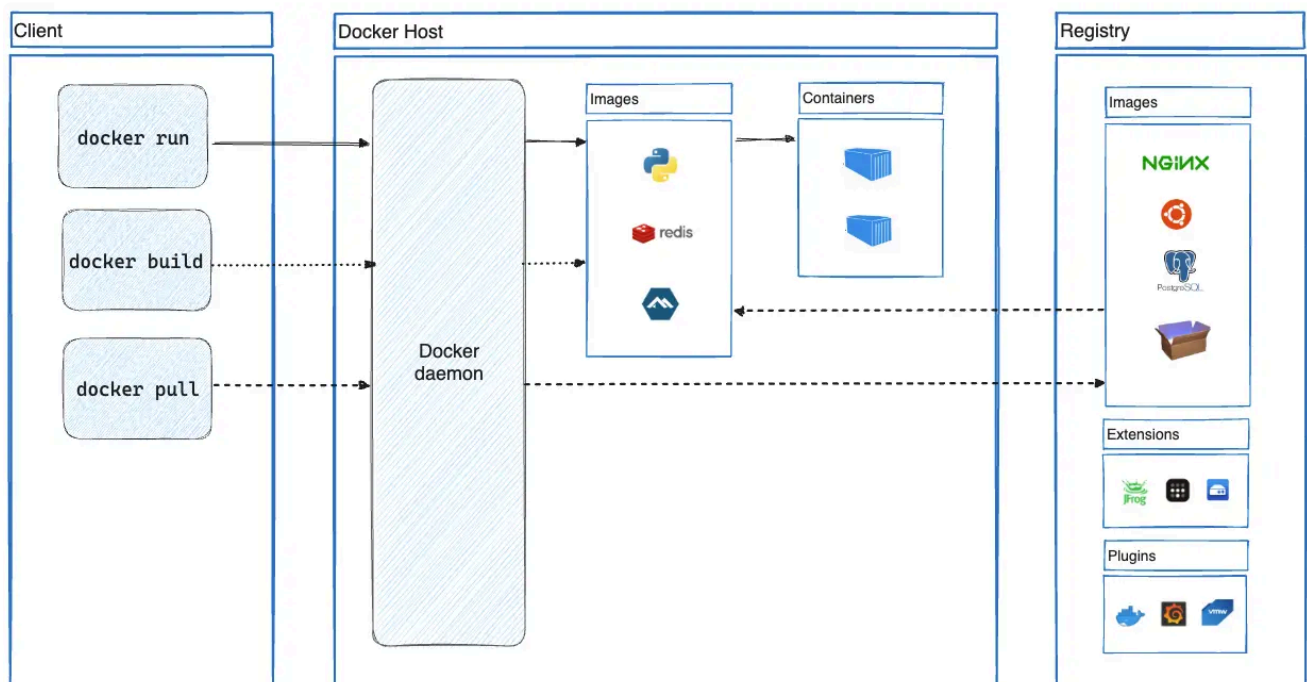
Docker's portability and lightweight nature also make it easy to dynamically manage workloads, scaling up or tearing down applications and services as business needs dictate, in near real time.

Running more workloads on the same hardware

Docker is lightweight and fast. It provides a viable, cost-effective alternative to hypervisor-based virtual machines, so you can use more of your server capacity to achieve your business goals. Docker is perfect for high density environments and for small and medium deployments where you need to do more with fewer resources.

Docker architecture

Docker uses a client-server architecture. The Docker client talks to the Docker daemon, which does the heavy lifting of building, running, and distributing your Docker containers. The Docker client and daemon can run on the same system, or you can connect a Docker client to a remote Docker daemon. The Docker client and daemon communicate using a REST API, over UNIX sockets or a network interface. Another Docker client is Docker Compose, that lets you work with applications consisting of a set of containers.



The Docker daemon

The Docker daemon (`dockerd`) listens for Docker API requests and manages Docker objects such as images, containers, networks, and volumes. A daemon can also communicate with other daemons to manage Docker services.

The Docker client

The Docker client (`docker`) is the primary way that many Docker users interact with Docker. When you use commands such as `docker run` , the client sends these commands to `dockerd` , which carries them out. The `docker` command uses the Docker API. The Docker client can communicate with more than one daemon.

Docker Desktop

Docker Desktop is an easy-to-install application for your Mac, Windows, or Linux environment that enables you to build and share containerized applications and microservices. Docker Desktop includes the Docker daemon (`dockerd`), the Docker client (`docker`), Docker Compose, Docker Content Trust, Kubernetes, and Credential Helper. For more information, see [Docker Desktop](#).

Docker registries

A Docker registry stores Docker images. Docker Hub is a public registry that anyone can use, and Docker looks for images on Docker Hub by default. You can even run your own private registry.

When you use the `docker pull` or `docker run` commands, Docker pulls the required images from your configured registry. When you use the `docker push` command, Docker pushes your image to your configured registry.

Docker objects

When you use Docker, you are creating and using images, containers, networks, volumes, plugins, and other objects. This section is a brief overview of some of those objects.

Images

An image is a read-only template with instructions for creating a Docker container. Often, an image is based on another image, with some additional customization. For example, you may build an image that is based on the Ubuntu image but includes the Apache web server

and your application, as well as the configuration details needed to make your application run.

You might create your own images or you might only use those created by others and published in a registry. To build your own image, you create a Dockerfile with a simple syntax for defining the steps needed to create the image and run it. Each instruction in a Dockerfile creates a layer in the image. When you change the Dockerfile and rebuild the image, only those layers which have changed are rebuilt. This is part of what makes images so lightweight, small, and fast, when compared to other virtualization technologies.

Containers

A container is a runnable instance of an image. You can create, start, stop, move, or delete a container using the Docker API or CLI. You can connect a container to one or more networks, attach storage to it, or even create a new image based on its current state.

By default, a container is relatively well isolated from other containers and its host machine. You can control how isolated a container's network, storage, or other underlying subsystems are from other containers or from the host machine.

A container is defined by its image as well as any configuration options you provide to it when you create or start it. When a container is removed, any changes to its state that aren't stored in persistent storage disappear.

Example `docker run` command

The following command runs an `ubuntu` container, attaches interactively to your local command-line session, and runs `/bin/bash`.

```
$ docker run -i -t ubuntu /bin/bash
```

When you run this command, the following happens (assuming you are using the default registry configuration):

1. If you don't have the `ubuntu` image locally, Docker pulls it from your configured registry, as though you had run `docker pull ubuntu` manually.
2. Docker creates a new container, as though you had run a `docker container create` command manually.

3. Docker allocates a read-write filesystem to the container, as its final layer. This allows a running container to create or modify files and directories in its local filesystem.
4. Docker creates a network interface to connect the container to the default network, since you didn't specify any networking options. This includes assigning an IP address to the container. By default, containers can connect to external networks using the host machine's network connection.
5. Docker starts the container and executes `/bin/bash`. Because the container is running interactively and attached to your terminal (due to the `-i` and `-t` flags), you can provide input using your keyboard while Docker logs the output to your terminal.
6. When you run `exit` to terminate the `/bin/bash` command, the container stops but isn't removed. You can start it again or remove it.

The underlying technology

Docker is written in the [Go programming language](#) and takes advantage of several features of the Linux kernel to deliver its functionality. Docker uses a technology called `namespaces` to provide the isolated workspace called the container. When you run a container, Docker creates a set of namespaces for that container.

These namespaces provide a layer of isolation. Each aspect of a container runs in a separate namespace and its access is limited to that namespace.

Next steps

- [Install Docker](#)
- [Get started with Docker](#)

Copyright © 2013-2026 Docker Inc. All rights reserved.

[Home](#) / [Get started](#) / [Introduction](#) / Build and push your first image

Build and push your first image

Table of contents

Explanation

Container images

Docker Hub

Try it out

Sign in with your Docker account

Create an image repository

Build and push the image

Recap

Next steps



How to Build and Push Your First Docker Image | Docker Con
Docker



Watch on

Explanation

Now that you've updated the [to-do list app](#), you're ready to create a container image for the application and share it on Docker Hub. To do so, you will need to do the following:

1. Sign in with your Docker account
2. Create an image repository on Docker Hub
3. Build the container image
4. Push the image to Docker Hub

Before you dive into the hands-on guide, the following are a few core concepts that you should be aware of.

Container images

If you're new to container images, think of them as a standardized package that contains everything needed to run an application, including its files, configuration, and dependencies. These packages can then be distributed and shared with others.

Docker Hub

To share your Docker images, you need a place to store them. This is where registries come in. While there are many registries, Docker Hub is the default and go-to registry for images. Docker Hub provides both a place for you to store your own images and to find images from others to either run or use as the bases for your own images.

When choosing base images, Docker Hub offers two categories of trusted, Docker-maintained images:

- [Docker Official Images \(DOI\)](#) – Curated images for popular software, following best practices and regularly updated.
- [Docker Hardened Images \(DHI\)](#) – Minimal, secure, production-ready images with near-zero CVEs, designed to reduce attack surface and simplify compliance. DHI images are free and open source under Apache 2.0.

In [Develop with containers](#), you used the following images that came from Docker Hub, each of which are [Docker Official Images](#):

- [node](#) - provides a Node environment and is used as the base of your development efforts. This image is also used as the base for the final application image.
- [mysql](#) - provides a MySQL database to store the to-do list items
- [phpmyadmin](#) - provides phpMyAdmin, a web-based interface to the MySQL database
- [traefik](#) - provides Traefik, a modern HTTP reverse proxy and load balancer that routes requests to the appropriate container based on routing rules

Explore the full catalog of trusted content on Docker Hub:

- [Docker Official Images](#) – Curated images for popular software
- [Docker Hardened Images](#) – Security-hardened, minimal production images (also available at [dhi.io](#))
- [Docker Verified Publishers](#) – Images from verified software vendors
- [Docker Sponsored Open Source](#) – Images from sponsored OSS projects

Try it out

In this hands-on guide, you'll learn how to sign in to Docker Hub and push images to Docker Hub repository.

Sign in with your Docker account

To push images to Docker Hub, you will need to sign in with a Docker account.

1. Open the Docker Dashboard.
2. Select **Sign in** at the top-right corner.
3. If needed, create an account and then complete the sign-in flow.

Once you're done, you should see the **Sign in** button turn into a profile picture.

Create an image repository

Now that you have an account, you can create an image repository. Just as a Git repository holds source code, an image repository stores container images.

1. Go to [Docker Hub](#).

2. Select **Create repository**.
3. On the **Create repository** page, enter the following information:
 - **Repository name** - `getting-started-todo-app`
 - **Short description** - feel free to enter a description if you'd like
 - **Visibility** - select **Public** to allow others to pull your customized to-do app
4. Select **Create** to create the repository.

Build and push the image

Now that you have a repository, you are ready to build and push your image. An important note is that the image you are building extends the Node image, meaning you don't need to install or configure Node, yarn, etc. You can simply focus on what makes your application unique.

What is an image/Dockerfile?

Without going too deep yet, think of a container image as a single package that contains everything needed to run a process. In this case, it will contain a Node environment, the backend code, and the compiled React code.

Any machine that runs a container using the image, will then be able to run the application as it was built without needing anything else pre-installed on the machine.

A `Dockerfile` is a text-based script that provides the instruction set on how to build the image. For this quick start, the repository already contains the `Dockerfile`.

CLI VS Code

1. To get started, either clone or [download the project as a ZIP file](#) to your local machine.

```
$ git clone https://github.com/docker/getting-started-todo-app
```

And after the project is cloned, navigate into the new directory created by the clone:

```
$ cd getting-started-todo-app
```

2. Build the project by running the following command, swapping out `DOCKER_USERNAME` with your username.

```
$ docker build -t DOCKER_USERNAME/getting-started-todo-app .
```

For example, if your Docker username was `mobydock`, you would run the following:

```
$ docker build -t mobydock/getting-started-todo-app .
```

3. To verify the image exists locally, you can use the `docker image ls` command:

```
$ docker image ls
```

You will see output similar to the following:

REPOSITORY	TAG	IMAGE ID	CREATED
mobydock/getting-started-todo-app	latest	1543656c9290	2 minutes ago
...			



4. To push the image, use the `docker push` command. Be sure to replace `DOCKER_USERNAME` with your username:

```
$ docker push DOCKER_USERNAME/getting-started-todo-app
```

Depending on your upload speeds, this may take a moment to push.

Recap

Before you move on, take a moment and reflect on what happened here. Within a few moments, you were able to build a container image that packages your application and push it to Docker Hub.

Going forward, you'll want to remember that:

- Docker Hub is the go-to registry for finding trusted content. Docker provides a collection of trusted content, composed of Docker Official Images, Docker Verified Publishers, and Docker Sponsored Open Source Software, to use directly or as bases for your own images.
- Docker Hub provides a marketplace to distribute your own applications. Anyone can create an account and distribute images. While you are publicly distributing the image you created, private repositories can ensure your images are accessible to only authorized users.

Usage of other registries

While Docker Hub is the default registry, registries are standardized and made interoperable through the [Open Container Initiative](#). This allows companies and organizations to run their own private registries. Quite often, trusted content is mirrored (or copied) from Docker Hub into these private registries.

Next steps

Now that you've built an image, it's time to discuss why you as a developer should learn



What's Next

Copyright © 2013-2026 Docker Inc. All rights reserved.



[Home](#) / [Get started](#) / [Docker workshop](#) / Part 8: Image-building best practices

Image-building best practices

Table of contents

[Image layering](#)

[Layer caching](#)

[Multi-stage builds](#)

[Maven/Tomcat example](#)

[React example](#)

[Summary](#)

[Next steps](#)

Image layering

Using the `docker image history` command, you can see the command that was used to create each layer within an image.

1. Use the `docker image history` command to see the layers in the `getting-started` image you created.

```
$ docker image history getting-started
```

You should get output that looks something like the following.

IMAGE	CREATED	CREATED BY
a78a40cbf866	18 seconds ago	/bin/sh -c #(nop) CMD ["node" "src/
f1d1808565d6	19 seconds ago	/bin/sh -c npm install --omit=dev
a2c054d14948	36 seconds ago	/bin/sh -c #(nop) COPY dir:5dc710ad8
9577ae713121	37 seconds ago	/bin/sh -c #(nop) WORKDIR /app

```

b95baba1cfdb      13 days ago      /bin/sh -c #(nop)  CMD ["node"]
<missing>         13 days ago      /bin/sh -c #(nop)  ENTRYPOINT ["dock
<missing>         13 days ago      /bin/sh -c #(nop)  COPY file:23873730
<missing>         13 days ago      /bin/sh -c apk add --no-cache --virt
<missing>         13 days ago      /bin/sh -c addgroup -g 1000 node
<missing>         13 days ago      /bin/sh -c #(nop)  ENV NODE_VERSION=
<missing>         13 days ago      /bin/sh -c #(nop)  CMD ["/bin/sh"]
<missing>         13 days ago      /bin/sh -c #(nop)  ADD file:e69d441d7

```

Each of the lines represents a layer in the image. The display here shows the base at the bottom with the newest layer at the top. Using this, you can also quickly see the size of each layer, helping diagnose large images.

2. You'll get the full output.

```
$ docker image history --no-trunc getting-started
```

Layer caching

Now that you've seen the layering in action, there's an important lesson to learn to help decrease build times for your container images. Once a layer changes, all downstream layers have to be recreated as well.

Look at the following Dockerfile you created for the getting started app.

```

# syntax=docker/dockerfile:1
FROM node:24-alpine
WORKDIR /app
COPY . .
RUN npm install --omit=dev
CMD ["node", "src/index.js"]
EXPOSE 3000

```

Going back to the image history output, you see that each command in the Dockerfile becomes a new layer in the image. You might remember that when you made a change to

the image, the dependencies had to be reinstalled. It doesn't make much sense to ship around the same dependencies every time you build.

To fix it, you need to restructure your Dockerfile to help support the caching of the dependencies. For Node-based applications, those dependencies are defined in the `package.json` file. You can copy only that file in first, install the dependencies, and then copy in everything else. Then, you only recreate the dependencies if there was a change to the `package.json`.

1. Update the Dockerfile to copy in the `package.json` first, install dependencies, and then copy everything else in.

```
# syntax=docker/dockerfile:1
FROM node:24-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm install --omit=dev
COPY . .
CMD ["node", "src/index.js"]
```

2. Build a new image using `docker build`.

```
$ docker build -t getting-started .
```

You should see output like the following.

```
[+] Building 16.1s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 175B
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load metadata for docker.io/library/node:24-alpine
=> [internal] load build context
=> => transferring context: 53.37MB
=> [1/5] FROM docker.io/library/node:24-alpine
=> CACHED [2/5] WORKDIR /app
=> [3/5] COPY package.json package-lock.json ./
=> [4/5] RUN npm install --omit=dev
```

```
=> [5/5] COPY . .  
=> exporting to image  
=> => exporting layers  
=> => writing image sha256:d6f819013566c54c50124ed94d5e66c452325327217f4  
=> => naming to docker.io/library/getting-started
```

3. Now, make a change to the `src/static/index.html` file. For example, change the `<title>` to "The Awesome Todo App".
4. Build the Docker image now using `docker build -t getting-started .` again. This time, your output should look a little different.

```
[+] Building 1.2s (10/10) FINISHED  
=> [internal] load build definition from Dockerfile  
=> => transferring dockerfile: 37B  
=> [internal] load .dockerignore  
=> => transferring context: 2B  
=> [internal] load metadata for docker.io/library/node:24-alpine  
=> [internal] load build context  
=> => transferring context: 450.43kB  
=> [1/5] FROM docker.io/library/node:24-alpine  
=> CACHED [2/5] WORKDIR /app  
=> CACHED [3/5] COPY package.json package-lock.json ./  
=> CACHED [4/5] RUN npm install  
=> [5/5] COPY . .  
=> exporting to image  
=> => exporting layers  
=> => writing image sha256:91790c87bcb096a83c2bd4eb512bc8b134c757cda0bde  
=> => naming to docker.io/library/getting-started
```

First off, you should notice that the build was much faster. And, you'll see that several steps are using previously cached layers. Pushing and pulling this image and updates to it will be much faster as well.

Multi-stage builds

Multi-stage builds are an incredibly powerful tool to help use multiple stages to create an image. There are several advantages for them:

- Separate build-time dependencies from runtime dependencies
- Reduce overall image size by shipping only what your app needs to run

Maven/Tomcat example

When building Java-based applications, you need a JDK to compile the source code to Java bytecode. However, that JDK isn't needed in production. Also, you might be using tools like Maven or Gradle to help build the app. Those also aren't needed in your final image. Multi-stage builds help.

```
# syntax=docker/dockerfile:1
FROM maven AS build
WORKDIR /app
COPY . .
RUN mvn package

FROM tomcat
COPY --from=build /app/target/file.war /usr/local/tomcat/webapps
```

In this example, you use one stage (called `build`) to perform the actual Java build using Maven. In the second stage (starting at `FROM tomcat`), you copy in files from the `build` stage. The final image is only the last stage being created, which can be overridden using the `--target` flag.

React example

When building React applications, you need a Node environment to compile the JS code (typically JSX), SASS stylesheets, and more into static HTML, JS, and CSS. If you aren't doing server-side rendering, you don't even need a Node environment for your production build. You can ship the static resources in a static nginx container.

```
# syntax=docker/dockerfile:1
FROM node:24-alpine AS build
WORKDIR /app
COPY package* ./
RUN npm install
COPY public ./public
COPY src ./src
```

```
RUN npm run build  
  
FROM nginx:alpine  
COPY --from=build /app/build /usr/share/nginx/html
```

In the previous Dockerfile example, it uses the `node:24-alpine` image to perform the build (maximizing layer caching) and then copies the output into an nginx container.

Tips

This React example is for illustration purposes. The getting-started todo app is a `Node.js` backend application, not a React frontend.

Summary

In this section, you learned a few image building best practices, including layer caching and multi-stage builds.

Related information:

- [Dockerfile reference](#)
- [Dockerfile best practices](#)

Next steps

In the next section, you'll learn about additional resources you can use to continue learning about containers.

What next

Copyright © 2013-2026 Docker Inc. All rights reserved.



[Home](#) / [Get started](#) / [Introduction](#) / [Develop with containers](#)

Develop with containers

Table of contents

Explanation

Try it out

Start the project

What's in the environment?

Make changes to the app

Change the greeting

Change the placeholder text

Change the background color

Recap

Next steps



Develop with Containers | Docker Concepts
Docker



Watch on

Explanation

Now that you have Docker Desktop installed, you are ready to do some application development. Specifically, you will do the following:

1. Clone and start a development project
2. Make changes to the backend and frontend
3. See the changes immediately

Try it out

In this hands-on guide, you'll learn how to develop with containers.

Start the project

1. To get started, either clone or [download the project as a ZIP file](#) to your local machine.

```
$ git clone https://github.com/docker/getting-started-todo-app
```

And after the project is cloned, navigate into the new directory created by the clone:

```
$ cd getting-started-todo-app
```

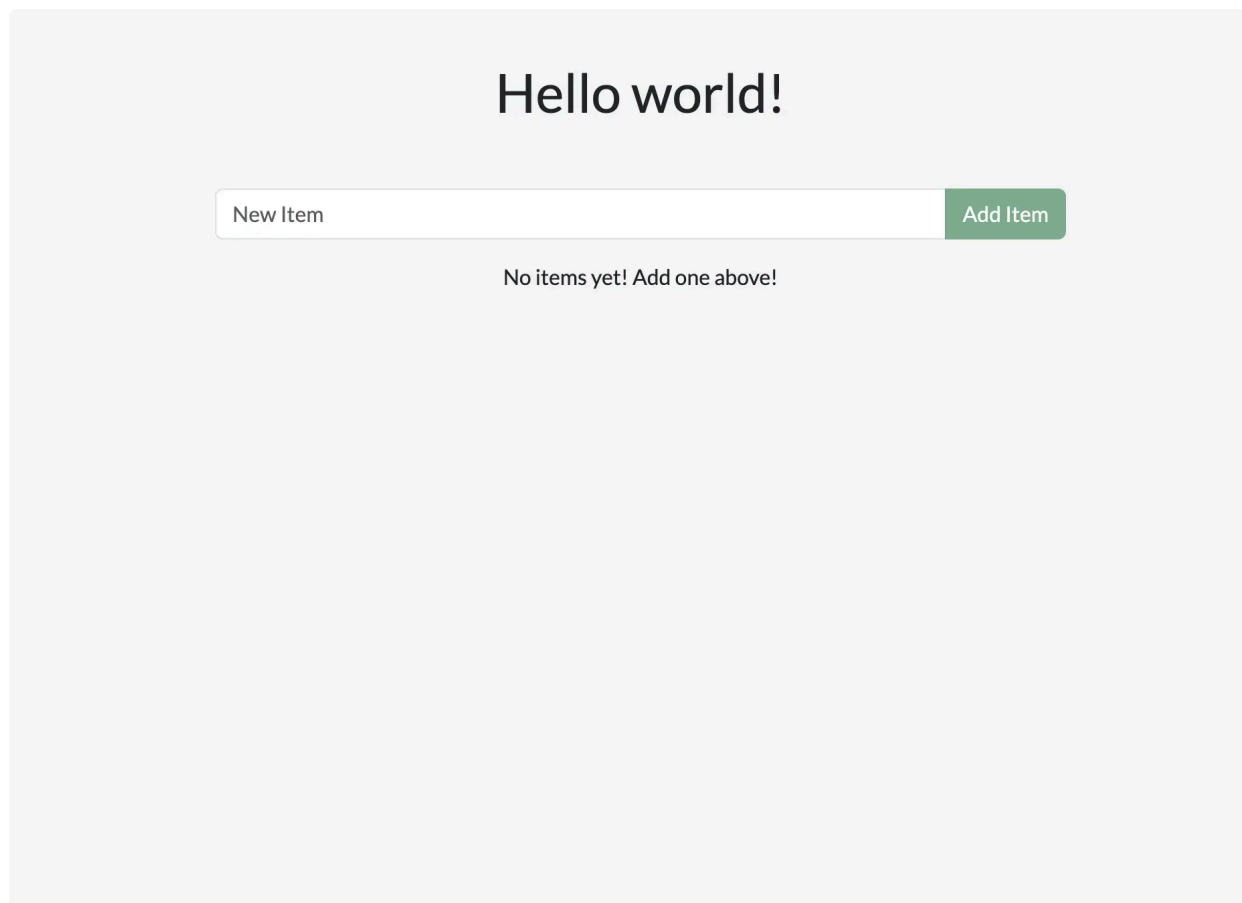
2. Once you have the project, start the development environment using Docker Compose.

To start the project using the CLI, run the following command:

```
$ docker compose watch
```

You will see an output that shows container images being pulled down, containers starting, and more. Don't worry if you don't understand it all at this point. But, within a moment or two, things should stabilize and finish.

3. Open your browser to <http://localhost> to see the application up and running. It may take a few minutes for the app to run. The app is a simple to-do application, so feel free to add an item or two, mark some as done, or even delete an item.



What's in the environment?

Now that the environment is up and running, what's actually in it? At a high-level, there are several containers (or processes) that each serve a specific need for the application:

- React frontend - a Node container that's running the React dev server, using [Vite](#).
- Node backend - the backend provides an API that provides the ability to retrieve, create, and delete to-do items.
- MySQL database - a database to store the list of the items.
- phpMyAdmin - a web-based interface to interact with the database that is accessible at <http://db.localhost>.
- Traefik proxy - [Traefik](#) is an application proxy that routes requests to the right service. It sends all requests for `localhost/api/*` to the backend, requests for `localhost/*` to the frontend, and then requests for `db.localhost` to phpMyAdmin. This provides the ability to access all applications using port 80 (instead of different ports for each service).

With this environment, you as the developer don't need to install or configure any services, populate a database schema, configure database credentials, or anything. You only need Docker Desktop. The rest just works.

Make changes to the app

With this environment up and running, you're ready to make a few changes to the application and see how Docker helps provide a fast feedback loop.

Change the greeting

The greeting at the top of the page is populated by an API call at `/api/greeting`. Currently, it always returns "Hello world!". You'll now modify it to return one of three randomized messages (that you'll get to choose).

1. Open the `backend/src/routes/getGreeting.js` file in a text editor on your local machine (in the cloned project directory). This file provides the handler for the API endpoint. Your changes will automatically sync to the running container.
2. Modify the variable at the top to an array of greetings. Feel free to use the following modifications or customize it to your own liking. Also, update the endpoint to send a random greeting from this list.

```
1  const GREETINGS = [  
2    "Whalecome!",  
3    "All hands on deck!",  
4    "Charting the course ahead!",  
5  ];  
6  
7  module.exports = async (req, res) => {  
8    res.send({  
9      greeting: GREETINGS[Math.floor(Math.random() * GREETINGS.length)],  
10    });  
11  };
```

3. If you haven't done so yet, save the file. If you refresh your browser, you should see a new greeting. If you keep refreshing, you should see all of the messages appear.

Charting the course ahead!

Add Item

No items yet! Add one above!

Change the placeholder text

When you look at the app, you'll see the placeholder text is simply "New Item". You'll now make that a little more descriptive and fun. You'll also make a few changes to the styling of the app too.

1. Open the `client/src/components/AddNewItemForm.jsx` file in your local project directory. This provides the component to add a new item to the to-do list.
2. Modify the `placeholder` attribute of the `Form.Control` element to whatever you'd like to display.

```
33 <Form.Control
34   value={newItem}
35   onChange={(e) => setNewItem(e.target.value)}
36   type="text"
37   placeholder="What do you need to do?"
38   aria-label="New item"
39 />
```

3. Save the file and go back to your browser. You should see the change already hot-reloaded into your browser. If you don't like it, feel free to tweak it until it looks just right.

Charting the course ahead!

Add Item

No items yet! Add one above!

Change the background color

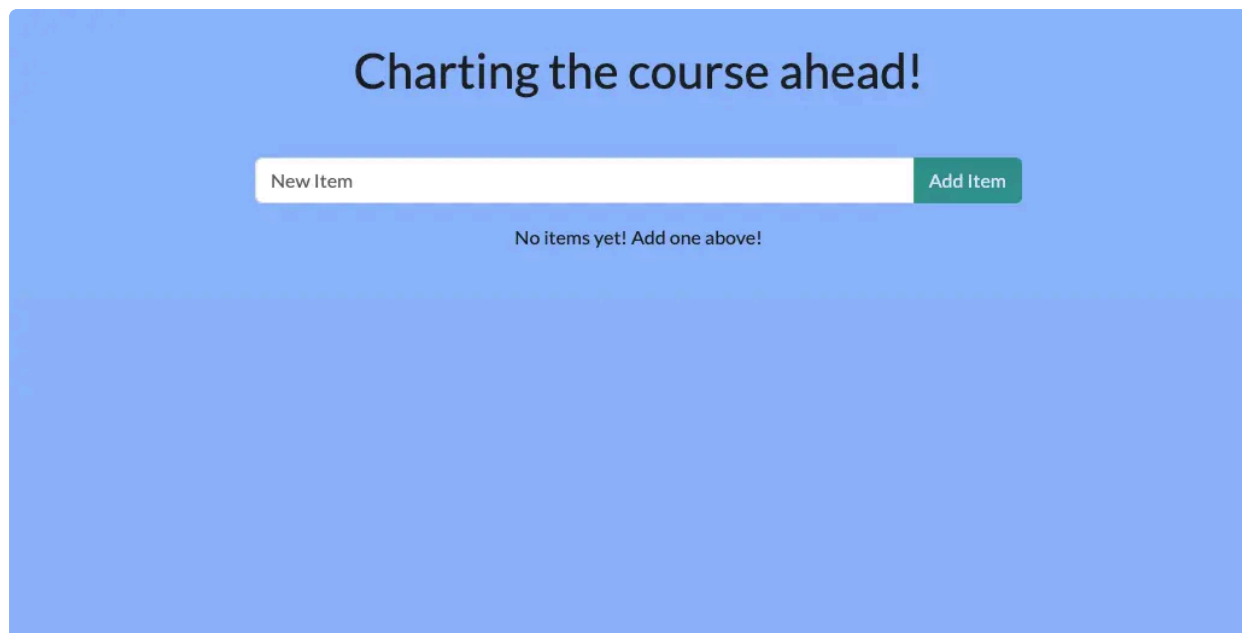
Before you consider the application finalized, you need to make the colors better.

1. Open the `client/src/index.scss` file in your local project directory.
2. Adjust the `background-color` attribute to any color you'd like. The provided snippet is a soft blue to go along with Docker's nautical theme.

If you're using an IDE, you can pick a color using the integrated color pickers. Otherwise, feel free to use an online [Color Picker](#).

```
3 body {  
4   background-color: #99bbff;  
5   margin-top: 50px;  
6   font-family: "Lato";  
7 }
```

Each save should let you see the change immediately in the browser. Keep adjusting it until it's the perfect setup for you.



And with that, you're done. Congrats on updating your website.

Recap

Before you move on, take a moment and reflect on what happened here. Within a few moments, you were able to:

- Start a complete development project with zero installation effort. The containerized environment provided the development environment, ensuring you have everything you need. You didn't have to install Node, MySQL, or any of the other dependencies directly on your machine. All you needed was Docker Desktop and a code editor.
- Make changes and see them immediately. This was made possible because
 1. the processes running in each container are watching and responding to file changes and 2) the files in your local project directory are shared with the containerized environment, so edits you make locally are automatically synced to the containers.

Docker Desktop enables all of this and so much more. Once you start thinking with containers, you can create almost any environment and easily share it with your team.

Next steps

Now that the application has been updated, you're ready to learn about packaging it as a container image and pushing it to a registry, specifically Docker Hub.

Build and push your first image

Copyright © 2013-2026 Docker Inc. All rights reserved.